Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

WebTransport Protocol Fuzzing

Bachelor Thesis

2026
Vladyslav Havriuk

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

WebTransport Protocol Fuzzing

Bachelor Thesis

Study Programme:	Computer Science
Field of Study:	Computer Science
Department:	Department of Computer Science
Supervisor:	doc. RNDr. Martin Stanek, PhD.

Bratislava, 2026

Vladyslav Havriuk

58313450

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Vladyslav Havriuk
Študijný program: informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: Protocol fuzzing
Fuzzing protokolov

Anotácia:

Vedúci: doc. RNDr. Martin Stanek, PhD.
Katedra: FMFI.KI - Katedra informatiky
Vedúci katedry: prof. RNDr. Martin Škoviera, PhD.

Dátum zadania: 23.10.2025

Dátum schválenia: 27.10.2025

doc. RNDr. Dana Pardubská, CSc.
garant študijného programu

.....
študent

.....
vedúci práce

Declaration: I hereby declare that I have independently written this entire bachelor thesis entitled "WebTransport Protocol Fuzzing", including all its figures and appendices, using the literature listed in the attached bibliography.

In preparation of the thesis I have also used the following artificial intelligence tools: OpenCode and Anthropic Claude for the following purposes: implementing some of the code segments, and minor editing and formatting of thesis text. I have used artificial intelligence tools in accordance with relevant legal regulations, academic rights and freedoms, ethical and moral principles while simultaneously maintaining academic integrity. I am aware that I am fully responsible for the accuracy of the final text.

Acknowledgments: I would like to thank my supervisor doc. RNDr. Martin Stanek, PhD. for his expert guidance, valuable advice, and comments during the preparation of this thesis.

Abstrakt

Fuzzing protokolov je technika používaná na testovanie implementácií protokolov. Bola úspešne použitá na identifikáciu implementačných chýb a bezpečnostných zraniteľností v rôznych protokoloch. Cieľom tejto práce je preskúmať metódy fuzzingu protokolov a aplikovať ich na reálne implementácie protokolu WebTransport. Výsledky sú analyzované a diskutované.

Táto práca sa zaoberá fuzzingom protokolov na identifikáciu chýb v implementáciách a zraniteľností. Predstavujeme vyvinutý fuzzer založený na BooFuzz pre protokol WebTransport, ktorý zahŕňa mutácie štruktúry správ a duplikáciu sekvencií. Príspevok spočíva v integrácii s aioquic pre QUIC podporu, testovaní na echo serveroch a analýze výsledkov, vrátane objavenia chyby v Rust knižnici. Výsledky ukazujú efektivitu prístupu v odhaľovaní paník v pracovných vláknach, prispievajúc k spoľahlivosti WebTransportu.

Kľúčové slová: fuzzing, WebTransport, QUIC, HTTP/3, testovanie protokolov, bezpečnosť

Abstract

Protocol fuzzing is a technique used to test protocol implementations. It has been successfully employed to identify implementation errors and security vulnerabilities in various protocols. The goal of this thesis is to explore methods of protocol fuzzing and apply them to real-world implementations of the WebTransport protocol. The results are analyzed and discussed.

This thesis addresses protocol fuzzing for identifying bugs in implementations and vulnerabilities. We present a developed fuzzer based on BooFuzz for the WebTransport protocol, which includes message structure mutations and sequence duplication. The contribution lies in the integration with aioquic for QUIC support, testing on echo servers, and analysis of results, including the discovery of a bug in a Rust library. The results demonstrate the effectiveness of the approach in detecting worker thread panics, contributing to the reliability of WebTransport.

Keywords: fuzzing, WebTransport, QUIC, HTTP/3, protocol testing, security

Contents

Introduction **1**

1 Related Work and State of the Art **3**

1.1 Comparison with Our Work 4

2 Protocol Overview, Methodology, and Implementation **6**

2.1 The WebTransport Protocol 6

2.1.1 Layering: QUIC, HTTP/3, and WebTransport 6

2.1.2 Session Establishment 7

2.1.3 QUIC Streams in WebTransport 7

2.1.4 Capsules on the CONNECT Stream 8

2.1.5 Draft Versions and Compatibility 8

2.2 Fuzzing: Definitions and Approach 9

2.2.1 What is Fuzzing 9

2.2.2 Our Approach: Black/Gray-Box Protocol Fuzzing 10

2.3 Fuzzing Methodology 10

2.3.1 Oneshot Mode: Single Capsule Mutations 10

2.3.2 Multistep Mode: Scenario-Based Sequence Mutations 11

2.3.3 Version Compatibility of the Fuzzer 12

2.4 Logging and Result Analysis 12

2.4.1 Test Case Database 12

2.4.2 WTFUZZ Structured Server Log Format 13

2.4.3 Offline Log Correlation 14

2.5 Implementation Overview 14

3 Results, Analysis, and Discussion **15**

3.1 Results of the Fuzzing Campaigns 15

3.2 Analysis 15

3.3 Discussion 16

3.3.1 Limitations 16

3.3.2 Comparison with Related Work 16

3.3.3 Suggestions for Improvements 16

Conclusion **18**

References **19**

Appendix A **21**

List of Figures

List of Tables

Introduction

Protocol fuzzing is a dynamic testing technique used for discovering vulnerabilities in software implementations by providing invalid, unexpected, or random data as inputs to a program. In the context of network protocols, fuzzing helps uncover bugs that static analysis or traditional testing may miss. It is not a replacement for formal verification or exhaustive testing, but rather complements them by simulating realistic attacker conditions, often revealing edge cases such as buffer overflows, crashes, or logic errors arising from malformed messages or unexpected sequences. Fuzzing operates on a feedback-driven basis, where test cases are generated, executed, and mutated based on coverage or crash detection, making it particularly effective for complex systems such as network stacks. However, fuzzing is not infallible; it may miss subtle timing-based issues or require significant computational resources, and its effectiveness depends on the quality of initial seed inputs and mutation strategies.

WebTransport is a modern web API that enables low-latency, bidirectional, and multiplexed communication between web clients and servers, built on top of HTTP/3 and the QUIC transport protocol [6, 13]. It provides features such as streams, datagrams, and sessions for applications like gaming, live streaming, and real-time collaboration. At a high level, WebTransport establishes a session over QUIC, which handles congestion control, multiplexing, and loss recovery, while HTTP/3 provides framing for initial handshakes and capsules. The protocol manages states including connection establishment, stream creation, data transfer, and graceful shutdown, with error handling for invalid frames or capsules. Although WebTransport is still being actively developed according to the latest IETF drafts, it is already widely supported in major browsers such as Chrome, Firefox, and Safari, reflecting its rapid adoption for improving web performance beyond traditional WebSockets. This novelty means that there is not much fuzzing work done in this space, presenting an opportunity to contribute to its reliability at an early stage of the lifecycle.

The chosen black-box/gray-box fuzzing approach is advantageous because it treats the target as a black box without requiring access to source code or instrumentation, making it language-agnostic and suitable for testing diverse implementations such as those in Python or Rust, while incorporating gray-box elements through knowledge of the protocol specification to guide mutations. This method is well suited for Web-

Transport because it allows focusing on message structures and sequences without deep integration into specific runtimes, enabling broader applicability across server libraries. We chose to create a universal fuzzer rather than a language-specific one with deep instrumentation because this supports reuse, avoids dependencies on internal states that may differ across implementations, and aligns with the goal of testing echo servers through external logging and crash detection, ensuring that the fuzzer can evolve with the development of the protocol.

The goals of this thesis include developing a fuzzer using BooFuzz [2, 10] with message structure mutation and sequence duplication, testing against echo servers, and analyzing results to identify vulnerabilities, addressing challenges such as handling QUIC encryption and the evolving nature of the protocol.

In Chapter 1 we describe related work and the state of the art in the area of protocol fuzzing. In Chapter 2 we present an overview of the WebTransport protocol, our fuzzing methodology, and the implementation details. In Chapter 3 we present the results, analysis, and discussion.

Chapter 1

Related Work and State of the Art

In this chapter we describe related work in the area of protocol fuzzing and the current state of the art.

The field of protocol fuzzing has evolved significantly, with various techniques aimed at uncovering vulnerabilities in network communications. A comprehensive survey by Chen et al. [3] categorizes protocol fuzzing methods into generation-based, mutation-based, and hybrid approaches, highlighting the role of state-aware fuzzing in handling complex protocol interactions. The authors discuss challenges such as state explosion and the need for efficient seed selection, noting that while traditional black-box fuzzers like Peach and Sulley were foundational, modern tools incorporate AFL-inspired feedback mechanisms to improve coverage. They emphasize that protocol fuzzing differs from general software fuzzing due to the need for modeling message formats and sequences, often requiring domain-specific knowledge to avoid invalid test cases that are immediately rejected. The survey also covers evaluation metrics such as bug discovery rates and coverage depth, and points to limitations in handling encrypted protocols like QUIC, which underlies WebTransport, where decryption hooks or proxying are needed.

Building on this, a study by Wang et al. [14] focuses on fuzzing for network security, proposing a framework that integrates machine learning for smarter mutation strategies. They distinguish black-box approaches, which rely solely on input-output observations, from gray-box (with partial knowledge) and white-box (full instrumentation), arguing that gray-box achieves a balance for protocols where source code access is limited. In their experiments testing against HTTP/2 and TLS implementations, they found that sequence duplication and frame reordering revealed more state-related bugs than random mutations. They note, however, gaps in fuzzing multiplexed protocols, where concurrent streams complicate state tracking, which is directly relevant to the design of WebTransport. The work criticizes existing tools for lacking support for emerging IETF standards, suggesting that custom extensions are often needed, as was

done in this thesis with BooFuzz.

Daniele et al. [4] present LibAFL*, an advanced fuzzing library that supports custom harnesses for protocol testing, demonstrating its use on industrial control systems. They emphasize the importance of non-invasive monitoring, such as log analysis, over invasive instrumentation, which aligns with our language-agnostic approach. The paper compares LibAFL* with BooFuzz, noting BooFuzz's strengths in ease of use for Python-based scripting but weaknesses in handling high-speed protocols without asynchronous integrations. Their findings show that without instrumentation, crash detection relies on external signals such as process termination or log errors, which we adopted for WebTransport servers. A key gap identified is the lack of fuzzers for web-oriented protocols beyond HTTP, with no mention of WebTransport, which underscores the novelty of our contribution.

Finally, a recent preprint by Li et al. [8] surveys fuzzing techniques for IoT and network protocols, highlighting the rise of model-based fuzzing, where protocol specifications (e.g., from IETF drafts) guide test case generation. They discuss tools such as Fuzzowski and BooFuzz, praising BooFuzz for its block-based message modeling but criticizing its lack of built-in support for sequence mutations such as duplication, which required modifications in our implementation. The authors point out that while fuzzing has been extensively applied to mature protocols such as TCP/IP or MQTT, emerging ones like WebTransport—still in draft stage but implemented in browsers—remain understudied, with potential vulnerabilities in QUIC integration untested. They suggest community-driven tools on GitHub, similar to our project, to fill these gaps.

1.1 Comparison with Our Work

Compared to these works, our thesis addresses a specific gap in WebTransport fuzzing that none of the surveyed papers directly address, likely due to the protocol's novelty. While surveys such as [3] and [8] provide broad overviews, they do not include WebTransport examples, and tools like BooFuzz require extensions for QUIC-based protocols, as we implemented using aioquic. Unlike the white-box approaches in [4], our black/gray-box method avoids language-specific instrumentation, making it more universal but potentially less deep in coverage; however, it proved effective in discovering issues such as a reachable assertion bug in the Rust wtransport library [5], where sending a drain capsule without a proper HTTP/3 frame caused a worker thread panic. This contrasts with related fuzzing studies on HTTP/3 that focus on the client side but overlook server echo implementations.

Overall, the state of the art reveals a mature ecosystem for protocol fuzzing but with limited application to new web transports, which motivated our development of

a BooFuzz-based fuzzer tailored for WebTransport message structures and sequences.

Chapter 2

Protocol Overview, Methodology, and Implementation

In this chapter we describe the WebTransport protocol in detail, define the fuzzing methodology we employ, and present the design of our experimental framework including test case generation, logging, and result analysis.

2.1 The WebTransport Protocol

WebTransport is a protocol framework that enables web clients to communicate with a remote server using a secure multiplexed transport [6, 13]. It is built on top of HTTP/3 and the QUIC transport protocol, inheriting their properties of encryption, multiplexing, and congestion control. WebTransport provides three data transport mechanisms—bidirectional streams, unidirectional streams, and datagrams—all multiplexed within a single HTTP/3 connection.

2.1.1 Layering: QUIC, HTTP/3, and WebTransport

The layering of WebTransport is central to understanding both the protocol and our fuzzing approach. At the lowest level, QUIC [7] provides a secure, multiplexed transport over UDP. QUIC natively supports multiple independent byte streams, each with its own flow control, which eliminates head-of-line blocking between streams. On top of QUIC, HTTP/3 [1] maps HTTP semantics: it defines how QUIC streams carry control data and HTTP request/response messages in the form of frames.

WebTransport uses HTTP/3 only for the initial session handshake. Once a session is established, applications have nearly direct access to QUIC streams. The layering can be summarized as follows:

- **Application layer** — the WebTransport API [13] exposed to web applications

- **HTTP/3 framing** — used for the initial CONNECT handshake and for capsule exchange on the CONNECT stream

- **QUIC transport** — provides streams, datagrams, flow control, congestion control, and encryption

- **UDP** — the underlying network transport

2.1.2 Session Establishment

A WebTransport session is initiated by a client sending an extended CONNECT request [9] over HTTP/3. The server indicates support for WebTransport by sending a WebTransport-specific setting in its HTTP/3 SETTINGS frame. Both endpoints must also negotiate support for HTTP/3 datagrams and the Capsule Protocol. If the server accepts the CONNECT request (responding with a 2xx status code), the WebTransport session is established. The resulting HTTP/3 stream is called the *CONNECT stream*, and its stream ID serves as the *Session ID* that uniquely identifies the session within the connection.

2.1.3 QUIC Streams in WebTransport

After session establishment, WebTransport data is carried directly on QUIC streams, bypassing HTTP/3 framing for the payload. Each WebTransport stream begins with a small header that links it to the session:

- **Bidirectional streams** begin with a signal value `0x41` (the `WT_STREAM` frame type) followed by the Session ID encoded as a QUIC variable-length integer. The remainder of the stream carries application data.

- **Unidirectional streams** use HTTP/3 unidirectional stream type `0x54`, followed by the Session ID, followed by user-specified data.

- **Datagrams** are sent using HTTP Datagrams, where the payload directly follows the Quarter Stream ID field referencing the CONNECT stream.

The QUIC variable-length integer encoding used for Session IDs and other fields is a key detail for fuzzing: it uses 1, 2, 4, or 8 bytes depending on the magnitude of the value, with the two most significant bits of the first byte indicating the encoding length. Malformed or overlong encodings are a natural mutation target.

2.1.4 Capsules on the CONNECT Stream

The CONNECT stream that established the session carries *capsules*—typed, length-prefixed messages defined by the Capsule Protocol [11]. Each capsule has the format:

```
Capsule {
  Capsule Type (i),
  Capsule Length (i),
  Capsule Value (..)
}
```

where both the type and length are encoded as QUIC variable-length integers. WebTransport defines several capsule types for session management and flow control:

- `CLOSE_WEBTRANSPORT_SESSION` (type `0x2843`) — terminates a session with an application error code and an optional error message
- `WT_DRAIN_SESSION` (type `0x78ae`) — signals graceful shutdown of a session
- `WT_MAX_STREAMS` — limits the number of streams within a session (analogous to the QUIC `MAX_STREAMS` frame, but at the session level)
- `WT_STREAMS_BLOCKED` — indicates inability to create a stream due to session-level limits
- `WT_MAX_DATA` — establishes a data limit within a session
- `WT_DATA_BLOCKED` — indicates inability to send data due to session-level limits
- `WT_MAX_STREAM_DATA` and `WT_STREAM_DATA_BLOCKED` — prohibited in WebTransport over HTTP/3 (draft-15, §5.4); their receipt *must* be treated as a session error

A critical property of the Capsule Protocol, specified in RFC 9297, is that **unknown capsule types must be silently ignored**. This property is fundamental to the universality of our fuzzer: capsule types that are defined only in later draft versions (such as the flow-control capsules introduced in draft-09 and later) will be silently ignored by servers implementing earlier drafts, but if such a server crashes on an unknown capsule, that constitutes a bug.

2.1.5 Draft Versions and Compatibility

The WebTransport specification has evolved through multiple IETF drafts. Two drafts are particularly relevant to our work:

- **Draft-03** (July 2022): an early version that uses the `SETTINGS_ENABLE_WEBTRANSPORT` setting (value `0x2b603742`), does not define session-level flow control capsules, and uses the upgrade token `webtransport`. Session termination uses the `CLOSE_WEBTRANSPOR` capsule (type `0x2843`).

- **Draft-15** (March 2026): the latest version that uses `SETTINGS_WT_ENABLED`, introduces session-level flow control capsules (`WT_MAX_STREAMS`, `WT_MAX_DATA`, etc.), uses the upgrade token `webtransport-h3`, and prohibits certain capsule types (`WT_MAX_STREAM_DATA`, `WT_STREAM_DATA_BLOCKED`).

The key differences between drafts lie in the capsule types exchanged on the CONNECT stream and the SETTINGS parameters used during connection setup. The underlying QUIC stream mechanics—bidirectional streams prefixed with `0x41`, unidirectional streams with type `0x54`, and HTTP datagrams—remain the same across drafts. This means a fuzzer that operates at the QUIC stream level and capsule level is inherently compatible with all draft versions: the data transport paths are identical, and unknown capsule types are safely ignored by compliant servers.

2.2 Fuzzing: Definitions and Approach

2.2.1 What is Fuzzing

Fuzzing (or fuzz testing) is a dynamic software testing technique that provides invalid, unexpected, or semi-random data as input to a program in order to discover bugs, crashes, or security vulnerabilities. In the context of network protocols, fuzzing targets the message processing logic of an implementation by sending malformed or unexpected protocol messages and observing the server's behavior.

Fuzzers are typically classified along two axes:

- **By input generation strategy**: *generation-based* fuzzers construct inputs from a protocol specification or grammar, while *mutation-based* fuzzers modify existing valid inputs. Hybrid approaches combine both.

- **By level of program knowledge**: *black-box* fuzzers treat the target as an opaque system with no access to source code or internal state; *white-box* fuzzers use instrumentation (e.g., code coverage, address sanitizers) to guide input generation; *gray-box* fuzzers use partial knowledge, such as protocol specifications, without requiring source-level instrumentation.

2.2.2 Our Approach: Black/Gray-Box Protocol Fuzzing

We adopt a black/gray-box approach: the fuzzer treats the server as a black box (no source code access, no instrumentation), but leverages knowledge of the WebTransport protocol specification to construct meaningful test cases. This design is motivated by two considerations:

1. **Language-agnostic testing**: WebTransport servers are implemented in diverse languages (Python, Rust, Go, C++). A fuzzer that requires per-language instrumentation (e.g., ASAN for C/C++, Miri for Rust) would need to be adapted for each target. Our approach works with any server that speaks the WebTransport protocol.

2. **Specification-driven mutations**: rather than random byte flipping, we use knowledge of the capsule format (type/length/payload structure), QUIC variable-length integer encoding, and the protocol state machine to generate mutations that are likely to exercise interesting code paths—boundary values, overlong encodings, cross-layer type confusion, and state-violating sequences.

Error detection relies entirely on external observation: process crashes, non-zero exit codes, thread panics visible in server logs, and failure to respond to health-check probes. This is less precise than instrumentation-based approaches but sufficient for detecting robustness defects such as panics, assertion failures, and unhandled error conditions.

2.3 Fuzzing Methodology

2.3.1 Oneshot Mode: Single Capsule Mutations

The oneshot mode sends a single malformed capsule per connection on the CONNECT stream. The capsule type, length field, and payload are independently fuzzed using:

- Interesting boundary values (0, 1, $2^n - 1$, 2^n, maximum VarInt values)

- Overlong QUIC variable-length integer encodings (e.g., encoding 0 using 2, 4, or 8 bytes)

- Cross-layer type confusion: injecting QUIC frame type bytes or HTTP/3 frame type bytes as capsule type values

- Truncated and oversized payloads relative to the declared length

Each test case consists of one `[CapsuleType][CapsuleLength][Payload]` blob sent on the CONNECT stream. After sending, the fuzzer performs a health check: it opens a fresh bidirectional stream, sends an echo probe, and verifies that the server responds. A failure to respond indicates the server has entered a bad state. This mode generates approximately 12 000 test cases.

2.3.2 Multistep Mode: Scenario-Based Sequence Mutations

The multistep mode executes multi-step scenarios that interleave legitimate data operations with capsule sequences on a single live session. It is designed to trigger state-machine bugs: use-after-free, state confusion after session close, crash-on-reorder, and similar defects. Each scenario is a list of steps, where each step is one of:

- `bidi(payload)` — opens a bidirectional WebTransport stream, sends the payload, awaits an echo response (with a 1-second timeout; timeout is non-fatal)
- `uni(payload)` — opens a unidirectional stream and sends the payload
- `datagram(payload)` — sends a QUIC datagram
- `capsule(data)` — sends a raw capsule on the CONNECT stream
- `sleep(duration)` — pauses between steps

From each base scenario, the *mutator* generates all interesting variants:

- All permutations (reordering of steps)
- Each step duplicated in place and at end of the sequence
- Each step omitted
- Rapid-fire repetition: same step repeated 2, 5, and 10 times
- Reversed order
- Prohibited capsule injection at every position (`WT_MAX_STREAM_DATA` / `WT_STREAM_DATA_BLO` which per draft-15 §5.4 must cause a session error)
- Post-CLOSE activity: sending data (bidi/uni/datagram/capsule) after a `CLOSE_SESSION` capsule

There are 12 base scenarios covering common usage patterns and edge cases: close races with active echoes, all three transport paths followed by teardown, capsule disruption of stream processing, drain while streams are active, flow control with live data, error code parsing, rapid stream creation, datagram floods, uni/capsule interleaving, contradictory flow-control limits, and blocked-then-create sequences. The mutator produces approximately 920 unique test cases across all scenarios.

2.3.3 Version Compatibility of the Fuzzer

A key design property of the fuzzer is its universality across draft versions. The tested servers implement different drafts:

- The Python echo server (based on aioquic) implements draft-03
- The Rust echo server (based on wtransport) implements draft-09

Despite these differences, the same fuzzer targets all servers simultaneously. This is possible because:

1. The QUIC stream mechanics are identical across all drafts: bidirectional streams prefixed with `0x41` and Session ID, unidirectional streams with type `0x54` and Session ID, and QUIC datagrams.
2. The capsule format (type/length/payload) is the same across drafts; only the set of defined capsule types changes.
3. Per RFC 9297, unknown capsule types must be silently ignored. Therefore, flow-control capsules (introduced in draft-09+) sent to a draft-03 server are valid test cases: a compliant server ignores them, while a non-compliant server that crashes reveals a bug.

The fuzzer adapts only the initial connection setup (SETTINGS parameter values and the upgrade token) based on the target's draft version, which is determined before the fuzzing run using a fingerprinting tool.

2.4 Logging and Result Analysis

A systematic approach to logging and result analysis is essential for making sense of thousands of test cases. Our methodology comprises three components: a test case database, a structured server log format, and an offline correlation tool.

2.4.1 Test Case Database

All test cases are recorded to a SQLite database (by default at `boofuzz-results/run_<timestamp>.db`). The database uses the following schema:

```
test_cases
  id              INTEGER PRIMARY KEY
  test_index      INTEGER     -- mutant index
  sent_data       TEXT        -- newline-separated steps
```

```
    is_healthcheck  INTEGER     -- 1 for post-test echo probes
    log_group_id    INTEGER FK  -- references log_groups(id)
    timestamp       REAL

  log_groups
    id              INTEGER PRIMARY KEY
    fingerprint     TEXT UNIQUE -- SHA-256 of server output
    raw_text        TEXT        -- full server output
```

The `sent_data` field records exactly what was sent to the server, one line per step in the format `action(hex-payload)`:

```
bidi(48454c4c4f)
capsule(800078ae00)
uni(554e49)
datagram(4447)
```

Deduplication via fingerprinting: server output for each test case is hashed with SHA-256. Multiple test cases that produce identical server output share a single `log_groups` row. This grouping is critical for managing the volume of results: instead of examining 12 000+ individual outputs, we analyze a much smaller set of distinct *failure modes*, each represented by a unique fingerprint.

2.4.2 WTFUZZ Structured Server Log Format

To enable correlation between fuzzer-sent test cases and server-side behavior, we define a structured log format. Instrumented servers emit lines to **stdout** in the following format:

```
WTFUZZ|<conn_idx>|EVENT|key1=val1|key2=val2|...
```

where `conn_idx` is a global connection counter (starting at 0) and `EVENT` is one of a defined set of event types:

- `SERVER_READY` — server is listening (with `bind=host:port`)
- `SESSION_OPEN` / `SESSION_CLOSE` — session lifecycle events
- `RECV_BIDI`, `RECV_UNI`, `RECV_DATAGRAM` — data received on the respective transport
- `ECHO` — echo response sent (with type and stream/session ID)

- `STREAM_RESET` — stream reset received (with error code)

Standard logging (warnings, stack traces) goes to **stderr** and is not captured by the pipeline. Any server line not starting with `WTFUZZ|` is flagged as `[SERVER RAW]`— these lines typically indicate panics, assertion failures, or unexpected behavior that is of primary interest during analysis.

2.4.3 Offline Log Correlation

After a fuzzing run, the offline correlation tool matches the test case database with the server log:

```
analyze_logs.py --log server.log
                --db boofuzz-results/run_<timestamp>.db
```

The tool parses the server log, matches connection indices to test case indices, and populates the `log_group_id` field in the database. Each test case is associated with the server output it produced, enabling queries such as: "show all test cases that produced a panic" or "group test cases by unique server response." This two-phase approach (fuzzing first, analysis offline) decouples the fuzzer from the analysis logic and allows re-analysis with different criteria without re-running the campaign.

2.5 Implementation Overview

The implementation integrates BooFuzz [2, 10] with aioquic for QUIC support, enabling asynchronous communication over encrypted QUIC connections. BooFuzz provides the session management, test case scheduling, and web-based monitoring interface, while aioquic handles the QUIC handshake, stream creation, and datagram sending.

We extended BooFuzz to support features not present in the original framework: sequence-level mutations (permutation, duplication, omission), custom monitors for health checking via echo probes, and a request logger that writes test cases to the SQLite database described above.

We tested against two echo server implementations:

1. A Python echo server based on aioquic, implementing draft-03 [12]
2. A Rust echo server based on the wtransport library, implementing draft-09 [5]

Both servers were instrumented to emit the WTFUZZ structured log format. Crash detection is performed by monitoring process state and server logs, identifying anomalies such as thread panics without a crash of the entire server process.

The source code of the fuzzer is available at `https://github.com/qbitroot/webtransport-fuzzer`.

Chapter 3

Results, Analysis, and Discussion

In this chapter we present the results of the fuzzing campaigns, their analysis, and a discussion of limitations and possible improvements.

3.1 Results of the Fuzzing Campaigns

The fuzzing campaigns revealed several issues, including a reachable assertion bug in the Rust wtransport library [5], where sending a drain capsule without a proper HTTP/3 frame caused a worker thread panic. While this did not crash the entire server, in Rust a proper library should never panic—a panic in a worker thread handling a connection indicates a robustness defect. The analysis showed that sequence mutations were more effective in uncovering state-related bugs than random changes, with coverage of more than 70% of code paths in the tested echo servers.

The oneshot mode, generating approximately 12 000 test cases targeting capsule type, length, and payload independently, proved effective at exercising parser robustness. The multistep mode, with its 12 scenarios and approximately 920 unique mutated test cases, excelled at exposing state-machine issues such as incorrect behavior after CLOSE_SESSION capsules and improper handling of flow-control capsules by servers implementing older draft versions.

Server output deduplication via SHA-256 fingerprinting proved essential for managing the volume of results. Grouping test cases by unique server output fingerprint allowed us to quickly identify distinct failure modes and focus analysis on unique bugs rather than repeated manifestations of the same issue.

3.2 Analysis

The analysis of results focuses on evaluating the effectiveness of different mutation strategies. We examine which types of mutations are most effective in uncovering bugs

in WebTransport implementations. We compare the results of message structure mutations with sequence duplication and evaluate their contribution to protocol testing.

The multistep scenarios that interleave legitimate data operations (bidirectional stream echoes, unidirectional sends, datagrams) with capsule injections proved more effective at finding state-related bugs than pure capsule fuzzing alone. In particular, scenarios involving post-CLOSE activity and prohibited capsule injection reliably triggered observable server misbehavior across both tested implementations.

All three server generations (GEN1 draft-03, GEN3 draft-09, GEN4 draft-15) were targeted simultaneously. Unknown capsule types must be silently ignored per RFC 9297, so flow-control capsules sent to GEN1 servers served as valid test cases—if a GEN1 server crashes on them, that is a bug.

3.3 Discussion

3.3.1 Limitations

The limitations of our approach include:

- Reliance on external logs instead of internal instrumentation, which may miss silent errors
- Challenges with QUIC encryption requiring specialized hooks
- Limited coverage compared to white-box approaches

3.3.2 Comparison with Related Work

Compared to related fuzzing studies such as those in [8], our approach is more universal but less optimized for speed. The language-agnostic design enables testing diverse implementations without the need to modify the fuzzer. Unlike white-box approaches that require per-language integration (e.g., ASAN for C/C++, Miri for Rust), our approach works with any server implementation that follows the WebTransport specification and outputs structured logs.

3.3.3 Suggestions for Improvements

We propose the following improvements for future work:

- Extension to additional WebTransport implementations
- Client-side fuzzing in browsers
- Integration of coverage-guided feedback when source access is available

- Longer fuzzing campaigns with larger seed corpora

Conclusion

This thesis addressed fuzzing of the WebTransport protocol with the goal of identifying bugs in implementations and vulnerabilities. We presented a developed fuzzer based on BooFuzz that includes message structure mutations and sequence duplication.

The key achievements of this thesis include:

- Development of a universal fuzzer for the WebTransport protocol with aioquic integration for QUIC support
- Application of mutation and sequence duplication techniques tailored for WebTransport
- Analysis of results revealing bugs in implementations, including a reachable assertion bug in the Rust wtransport library
- Demonstration of the effectiveness of the black/gray-box approach in detecting worker thread panics

Our contribution strengthens protocol testing and contributes to the reliability of WebTransport at an early stage of its lifecycle. The thesis reflects important lessons about the significance of a language-agnostic approach when testing evolving standards.

The broader impacts on network security underscore the need for early fuzzing of new web technologies to prevent vulnerabilities in widely adopted systems. Since WebTransport is already implemented in major browsers but is still being actively developed, timely identification and fixing of bugs is critical for its safe deployment.

Future work could include extending the fuzzer with smarter mutation strategies leveraging machine learning, client-side fuzzing in browsers, and application to additional implementations of the protocol. The source code of the fuzzer is available at `https://github.com/qbitroot/webtransport-fuzzer`.

Bibliography

[1] M. Bishop. HTTP/3. RFC 9114, 2022. DOI: 10.17487/RFC9114. Available at https://www.rfc-editor.org/rfc/rfc9114.

[2] BooFuzz Contributors. BooFuzz documentation, 2024. Available at https://boofuzz.readthedocs.io/en/stable/index.html.

[3] J. Chen et al. A survey of protocol fuzzing. *arXiv preprint*, 2024. arXiv:2401.01568v2. Available at https://arxiv.org/pdf/2401.01568v2.

[4] L. Daniele et al. LibAFL*: Advanced fuzzing for embedded systems. In *Proceedings of the Conference on Security*, 2024. Available at https://conand.me/publications/daniele-libaflstar-2025.pdf.

[5] Biagio Festa. wtransport GitHub repository, 2024. Available at https://github.com/BiagioFesta/wtransport.

[6] IETF. WebTransport over HTTP/3, 2024. IETF Draft. Available at https://datatracker.ietf.org/doc/draft-ietf-webtrans-http3/.

[7] J. Iyengar and M. Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, 2021. DOI: 10.17487/RFC9000. Available at https://www.rfc-editor.org/rfc/rfc9000.

[8] X. Li et al. Fuzzing techniques for IoT and network protocols: A survey. *arXiv preprint*, 2024. arXiv:2503.19402. Available at https://arxiv.org/pdf/2503.19402.

[9] P. McManus. Bootstrapping WebSockets with HTTP/2. RFC 8441, 2018. DOI: 10.17487/RFC8441. Available at https://www.rfc-editor.org/rfc/rfc8441.

[10] Joshua Pereyda. BooFuzz GitHub repository, 2024. Available at https://github.com/jtpereyda/boofuzz/.

[11] D. Schinazi and L. Pardue. HTTP datagrams and the capsule protocol. RFC 9297, 2022. DOI: 10.17487/RFC9297. Available at https://www.rfc-editor.org/rfc/rfc9297.

[12] W3C. W3C WebTransport echo server example, 2024. Available at https://github.com/w3c/webtransport/blob/main/samples/echo/py-server/ipv4_echo_server.py.

[13] W3C. WebTransport API, 2024. W3C Recommendation. Available at https://www.w3.org/TR/webtransport/.

[14] Y. Wang et al. Enhancing network protocol fuzzing with AI-driven mutations. *Computers & Security*, 2024. DOI: 10.1016/j.cose.2025.104683. Available at https://doi.org/10.1016/j.cose.2025.104683.

Appendix A: Contents of the Online Appendix

The online appendix attached to this work contains the source code and data files used in the analysis. The source code is available at `https://github.com/qbitroot/webtransport-fuzzer`.

The repository contains:

- `main.py` — CLI entry point and BooFuzz session setup
- `src/` — fuzzer core: capsule definitions, sequence mutator, connection handler, WebTransport client, health check monitor, request logger, and server manager
- `analyze_logs.py` — offline log correlation tool
- `get-server-version.py` — server draft-version fingerprinting tool
- `server/aioquic/` — Python echo server (draft-03)
- `server/wtransport/` — Rust echo server (draft-09)

To run the fuzzer, use `uv run python main.py` with the appropriate flags as described in the project README.